CS 3224
Operating Systems

Professor G. Sandoval

01

NYU

Which Operating System are you using on you laptop/tablet today?

Windows 0%

Mac OS 0%

iOS (iPad) 0%

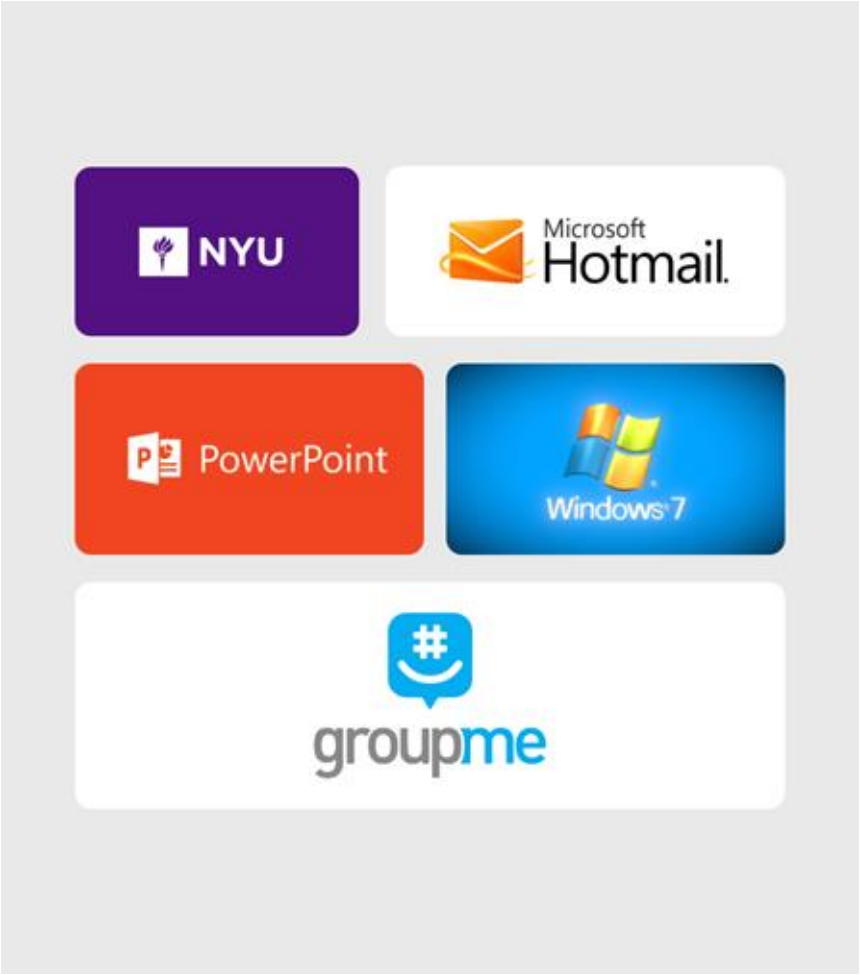
Linux 0%

None of the above 0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

About me

- 20 yrs at Microsoft
- 10 years at NYU
- Research: LLM safety, deep learning, and security



NYU

Why are you here? Why study OS?

- **Concurrency:** threads, locks, deadlocks, race conditions.
- **Resource Management**
- **Perf Analysis and Contention Resolution:** queuing, CPU Scheduling, prioritization
- **Interfaces and Hiding Complexity:** sockets, file systems, address spaces.
- **Dealing with Big Software**
- **Getting a Job!!**

Course Goals

- Understand **operating system design** and implementation
- Confidence with the **C programming** Language
- **Assembly language** Programming

NYU



Administrivia

How to get unblocked

- EdStem
- Integrated with Brightspace
- Use it for course questions and help

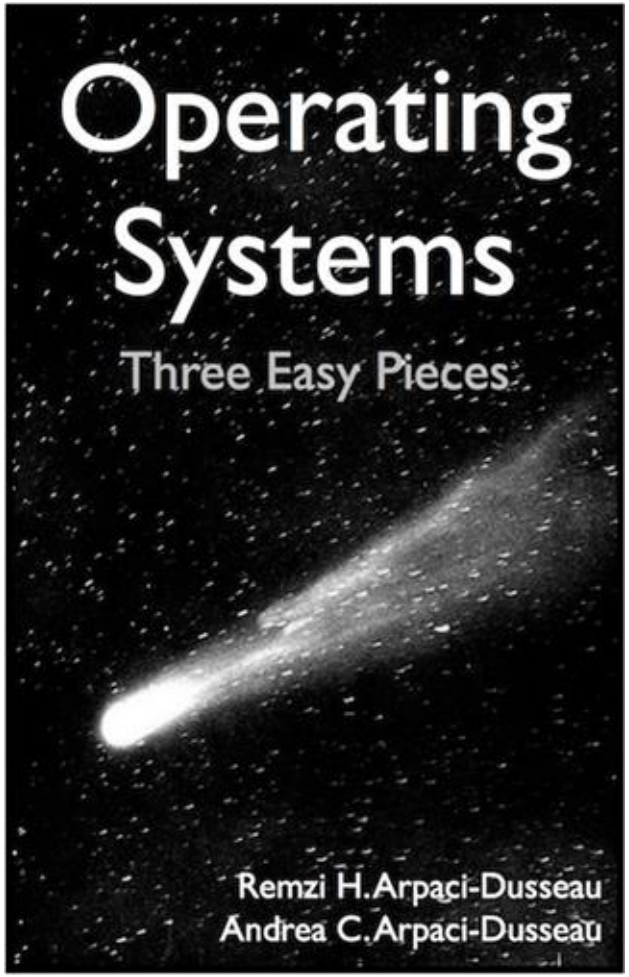
NYU

Homework

- Programming projects and questions from class
- Individual work unless stated otherwise
- Submit by the deadline; multiple submissions are allowed beforehand
- Anubis gives automated feedback in real time

Course materials

- OSTEP - free online; linked in Brightspace
- **xv6 book and source - posted in Brightspace**
- Modern Operating Systems - useful reference



NYU

C programming readiness

- Projects require C
- Experience with C++ helps, but C has important differences
- We will review C in lecture and recitation
- Use EdStem early if pointers, memory, or compilation feel unfamiliar

Grading

- **20%** Homework (Programming Projects)
- **35%** Midterm
- **35%** Final
- **10%** Participation (Polls 80% - >100%)

NYU

Late days

Each assignment has a 24-hour grace period

- After 24 hours, late penalties begin

NYU

Syllabus

- In Brightspace



NYU



(Very Quick) Computer history

Which came first?

- Put these in order:
- Plugboard programs
- Punched-card programs
- Stored programs
- Time-sharing
- Pair up - 60 seconds.

NYU

Pre-Pre-History

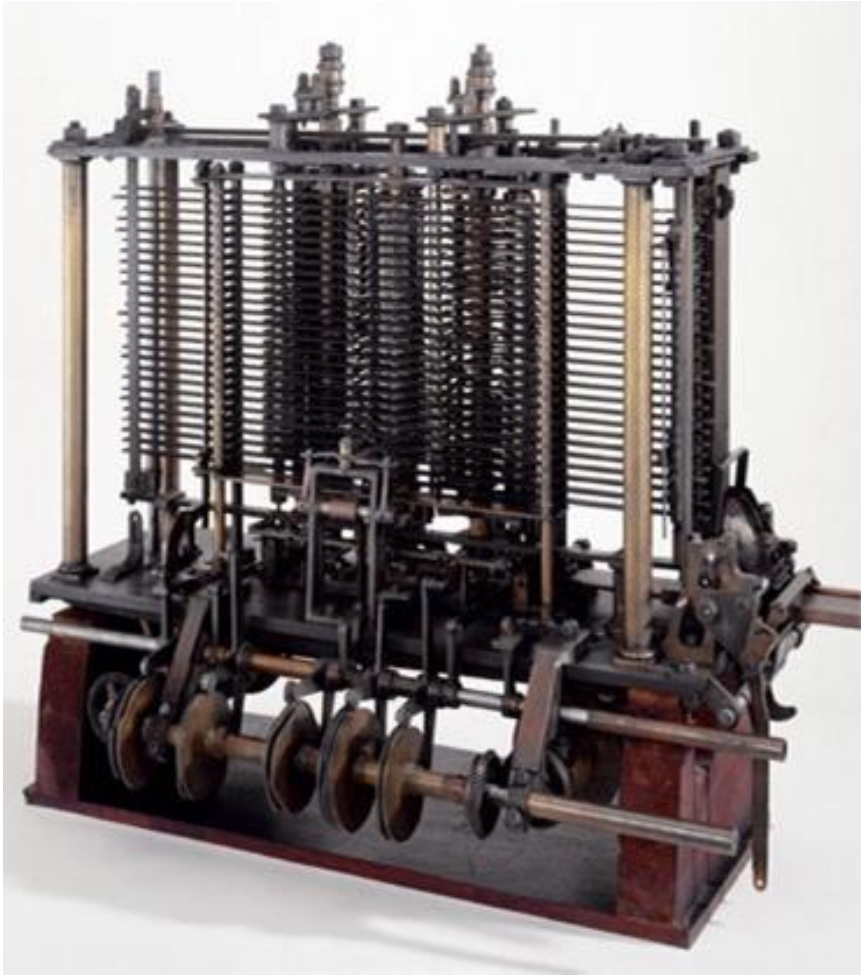
- The Jacquard Loom.
- Invented by Joseph Jacquard in 1801
- Used punched cards to describe patterns to weave
- We'll see such cards again…



NYU

The Analytical Engine (1837)

- Designed by Charles Babbage
- A general-purpose mechanical computer design
- Included arithmetic, conditional branching, and loops
- Never completed in Babbage's lifetime
    - Its design could express general computations



NYU

“Programs” on the IBM 402

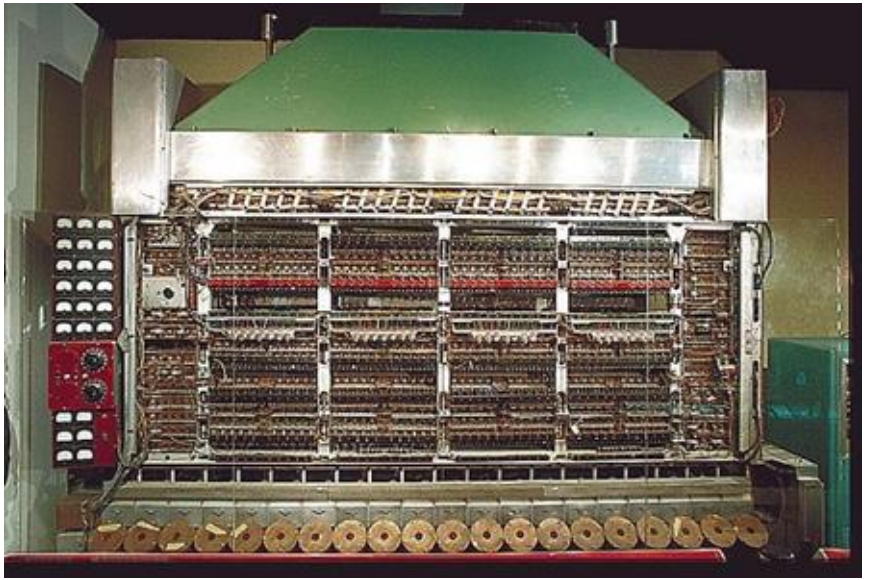
- Each program is actually a plugboard wired up to configure the machine
- This picture is from a company called Sparkler Filters, which still uses its IBM 402 from 1948



NYU

Institute for Advanced Study (IAS) Machine (1951)

- One of the earliest to store code and data in the same memory
- Origin of the term “von Neumann Bottleneck”



NYU

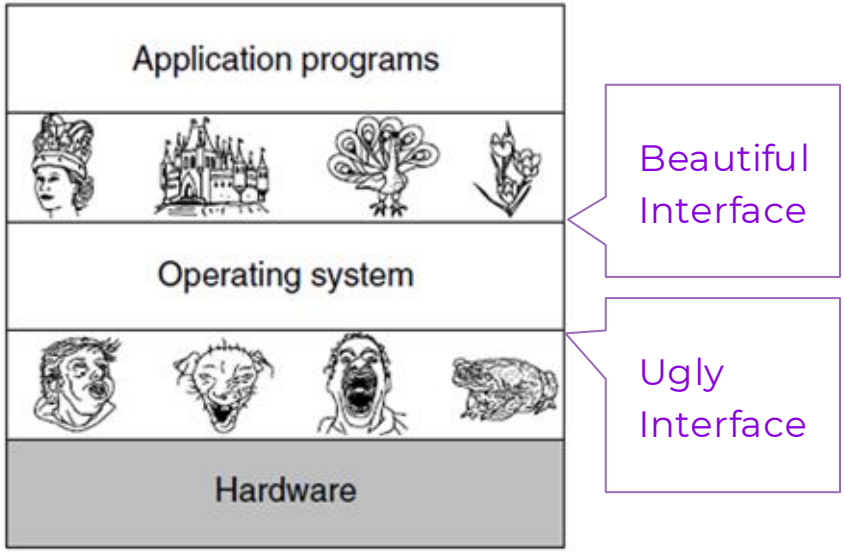
Early Computing History

- Early machines were programmed directly in machine language (or even direct wiring!)
  - No C, Java, Python, etc.
  - Not even any assembly language

NYU


NYU
What is an operating system?

The Operating System as an Extended Machine



Operating systems turn ugly hardware into beautiful abstractions.

NYU

Tanenbaum & Bo, Modern Operating Systems:4th ed.,
(c) 2013 Prentice-Hall, Inc. All rights reserved.

The Operating System as a Resource Manager

- **Top down view**
  - Provide abstractions to application programs
- **Bottom up view**
  - Manage pieces of complex system
- **Alternative view**
  - Provide orderly, controlled allocation of resources

NYU



History of operating systems

1. Vacuum Tube Technology

- Vacuum tubes acted as electronic switches
- Typical access time: about 100 ms
- "milli" means 10^-3; 100 ms = 0.1 s

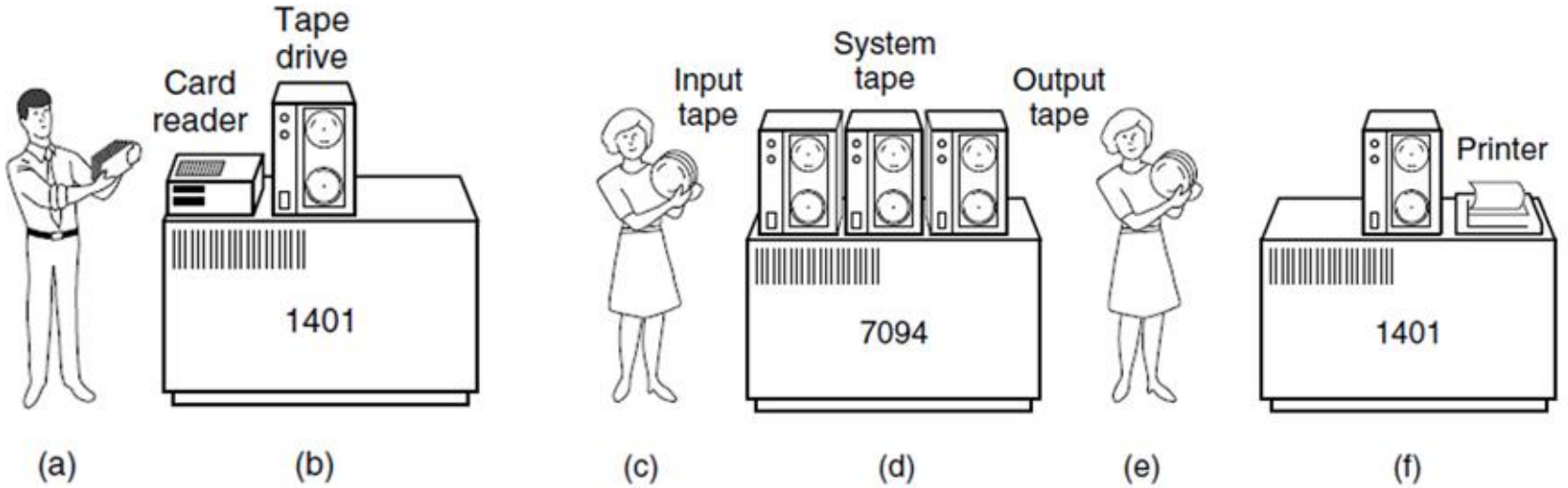


NYU

2. Transistor & Batch (1955-1965)

- Transistors made commercial computers faster and more reliable
- Computing time was expensive and usually metered
- Typical access time: about 100 microseconds
- 1 microsecond = 10^-6 seconds
- About 1,000x faster than 100 milliseconds

Batch jobs



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

ICs, multiprogramming, and time-sharing (1965-1980)

- Integrated circuits put many transistors on one chip
- IBM System/360 helped standardize a compatible computer family
- Multiprogramming kept the CPU busy by switching among jobs
- Time-sharing gave multiple users interactive access
- MULTICS influenced UNIX and its descendants

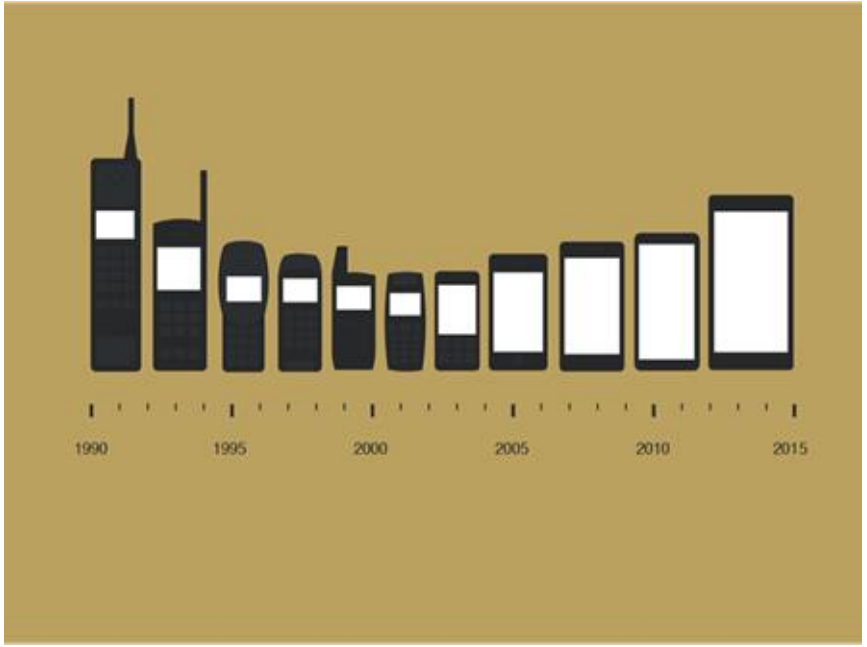
NYU

Personal computers (1980s onward)

- Large-scale integration made personal computers practical
- IBM PC (1981): Intel 8088 processor
- Microsoft DOS and Windows
- Apple II and Macintosh

Mobile computing (1990s onward)

- Operating systems moved into phones and tablets
- Symbian and BlackBerry OS -> iOS and Android
- New constraints: energy, sensors, radios, and mobility



NYU

Operating systems are everywhere

- Laptops and phones
- Cloud servers
- Cars, routers, and smart-home devices
- Wearables and specialized accelerators
- Which ones have you used today?



10-minute break

NYU



Intro to operating systems

Operating System: Three Easy Pieces

What happens on the CPU when a program runs?

- **The processor:**
  - **Fetches** an instruction from memory
  - **Decodes** the instruction
  - **Executes** the instruction
  - Moves on to the next instruction

Prediction: two programs, one CPU

- Both programs appear to run at once.
- What must the OS be doing?
- Think for 20 seconds, then compare with a neighbor.

NYU

An OS turns hardware into abstractions

Abstraction + resource management



Operating System (OS):

Software that turns hardware resources into useful abstractions for applications

The OS makes programs usable and safe

Run programs

- Share the processor and memory
- Communicate with devices and storage
- Protect applications from one another

Convenience, efficiency, and protection are all OS responsibilities.

Virtualization

- **The OS transforms a physical resource into a more useful virtual form**
  - **CPU -> processes**
  - Physical memory -> virtual address spaces
  - Storage device -> files and directories
  - Each application sees a simpler interface

OS as resource manager

- The OS allocates CPU time, memory, storage, and devices
- It balances performance, fairness, and protection

The Three Pieces

- 1. Virtualization - create useful illusions
- 2. Concurrency - coordinate simultaneous activity
- **3. Persistence - keep data beyond one program run**

NYU

Piece 1: Virtualization

- **Make each application behave as if it has resources to itself**
  - Examples: processes, virtual address spaces, and files

Virtualizing Memory (cont.)

- **Each process accesses its own private virtual address space.**
  - The OS maps **address space** onto the **physical memory**.
  - A memory reference within one running program **does not affect** the address space of other processes.
  - Physical memory is a **shared resource**, managed by the OS.

Two threads each run: counter = counter + 1 The counter starts at 0. Is the final value always 2?

yes

0%

No

0%

It depends.

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Piece 2: Concurrency

- **Multiple activities make progress and may interact**

  The hard part is preserving correctness when operations interleave

  - The OS provides mechanisms; programs must use them correctly

Power fails halfway through saving a file. After reboot, what should be allowed?

The old file? 0%

The new file? 0%

A half-updated file? 0%

None of the above 0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Piece 3: Persistence

- **Data outlives a process - and often a power cycle**
  - The OS provides files, directories, and links
  - The file system balances correctness, performance, and recovery

Persistence turns writes into durable state

- Choose where data will reside
  - Issue I/O requests to the storage device
  - **Recover from crashes with journaling or copy-on-write**

    Order writes carefully

NYU

OS design is a set of tradeoffs

- Convenience - useful abstractions
  - Performance - low overhead
- Protection - isolation between applications
  - Reliability - continue operating despite faults
  - Also: security, energy efficiency, and mobility

Why study Operating Systems?

OS knowledge travels across the stack

- Understand why workloads behave as they do
- Diagnose and tune performance
- Connect architecture, languages, data structures, and algorithms
    - Build confidence with large, complex systems

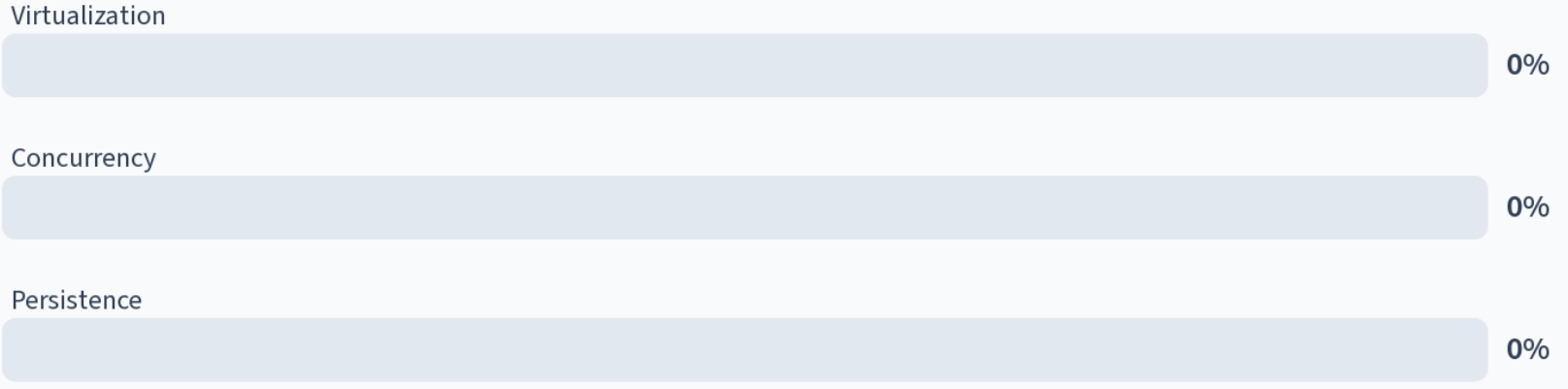
NYU

Exit ticket I

- In one sentence: What does an OS do?
- Name one useful illusion or abstraction.

NYU

Exit ticket: Which of the 3 pieces do you most want to understand:



Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**



Optional reference

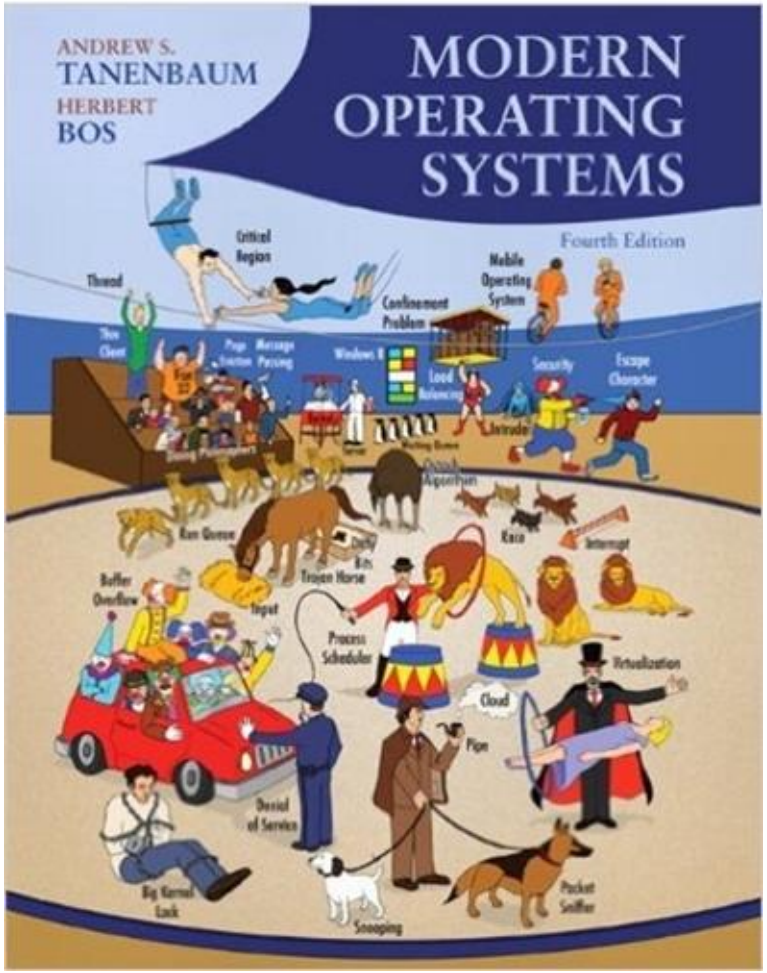
Textbook 1

- Throughout the semester we will be studying the source code of a UNIX-like operating system, **xv6**
- The authors of xv6 have written a short (100 page) book about its implementation, which accompanies the source code. You ***SHOULD*** read it.
- This has been **posted** on NYU Brightspace in the **Resources** section

NYU

Textbook 3

- Modern Operating Systems, by Andrew S. Tanenbaum and Herbert Bos
- Ebook/PDF and international editions are fine
- Available through Brightspace



NYU

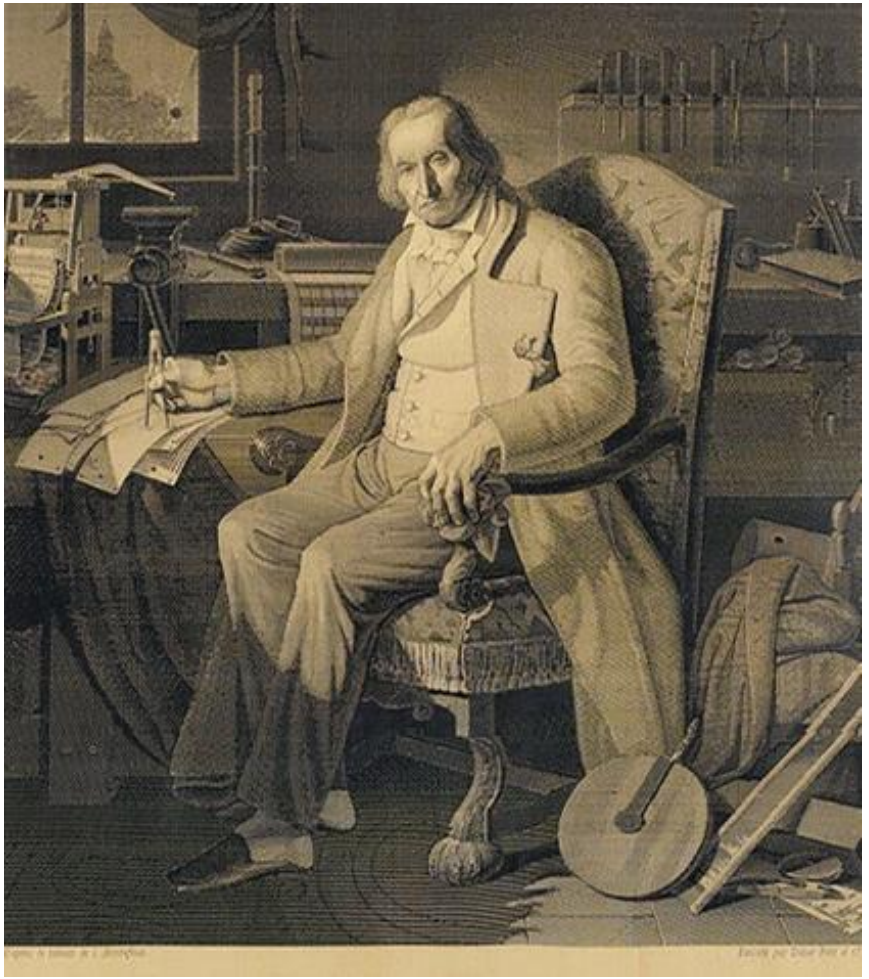
Optional recitation

- To help with Homework/Review
- Will depend on TA schedules
- Will be Taped
- Over zoom

NYU

Pre-Pre-History

- Jacquard Loom could even be used to weave portraits
- This one took 24,000 punch cards to specify
- A picture is worth 1,000 words, or 24,000 punch cards



NYU

Prehistoric Programming

- Ada Lovelace (1815-1852) worked with Babbage to create programs for the Analytical Engine
- Created an algorithm to compute Bernoulli numbers on the machine
- First Programmer and also the first programmer to stay up all night working on code!



NYU

Z3 Machine (1941)

- Electromechanical relay computer built in Germany by Konrad Zuse
- Technically Turing-complete (proved in 1998), but typically just carried out specific calculations

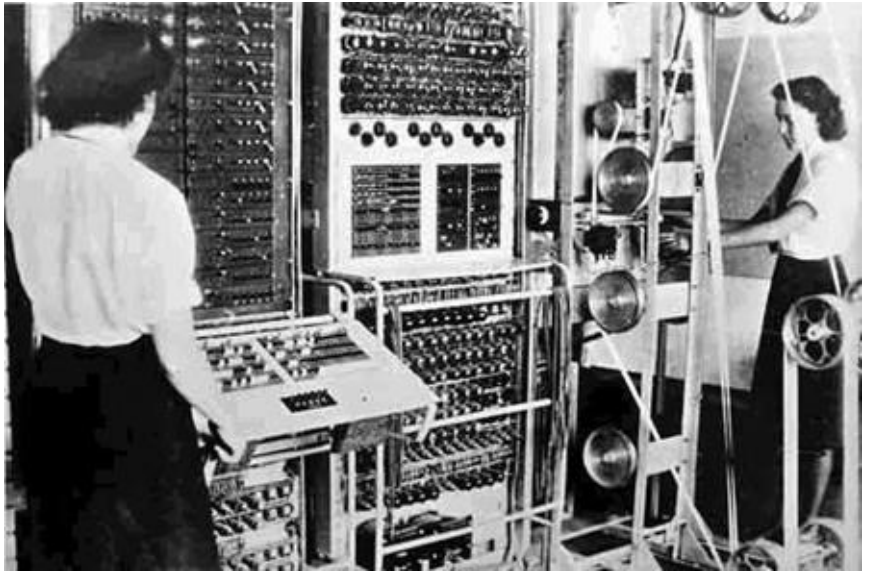


Photo by Wikipedia User Jahoe

NYU

Colossus (1943)

- Designed by Tommy Flowers at Bletchley Park (England) for use in decrypting German radio teleprinter messages (not Enigma)
- One of the programmers was Alan Turing



NYU

Harvard Mark I (1944)

- Used for atomic bomb computations in Manhattan Project
- Program “loops” by physically connecting the end of the program tape to the start, forming a loop
- One of it’s first programmers was Grace Hopper



Photo by Wikipedia User Topory

NYU

ENIAC (1946)

- Built at U Penn; used for artillery and hydrogen bomb computations



Photo by Wikipedia User TexasDex

Batch Jobs

- An individual job took a set of punch cards
- Cards were loaded onto magnetic tape (using a fast reader), carried over to the actual computer, and then the output tape was brought over to a printer
- The key idea of batch processing is to collect multiple jobs which can be loaded onto the tape and then processed all at once

Advanced Topics

- Current systems
  - Multiprocessors
  - Networked and distributed systems
  - Virtual machines

OS Design goals (cont.)

- High degree of reliability
  - The OS must also run non-stop
- Other issues:
  - Energy- efficiency
  - Security
  - Mobility